

Big Data Processing Case Study Report

COMP6579001 - Big Data Processing

2802457740 - Gerald Amadeo Passover Sutanto

2602163834 - George Bennett Winarto

2802482364 - Hansen Sebastian

2802472930 - Herrick Fabian

2802455230 - Ifan Ahnaf Sakhi Bakri

2602144646 - Jason Tanwinarta

2802465912 - Leonardo

2602145213 - Rafael Djefry Alvindja

2502087490 - Raihan Rosyada

2802474406 - Raymond Widjaja

2802522531 - Renjiro Wiroskid Angga

2802473712 - Samuel Alvincent

Air Quality Monitoring and Public Health

Contents

Cover Page.....	1
Executive Summary.....	2
A. Introduction.....	3
B. Big Data Architecture Design.....	4
C. Implementation Details.....	7
D. Result and Discussion.....	8
E. Conclusion.....	9
F. Declaration Statement.....	10
Reference.....	11

Executive Summary

The environment of Brooklyn faces a critical challenge about unpredictable fluctuations in PM2.5 air quality levels. Without a data-driven monitoring system, city officials struggle to issue health warnings, leaving local people exposed to sudden pollution spikes and increased respiratory risks. To address this problem, the project utilized the OPENAQ 2025 dataset, integrating historical sensor readings from August 2025. The analysis specifically relied on two dataset files: *measurements.csv*, for tracking daily pollution values and *stations.csv*, for the spatial metadata of localized monitoring sites.

To process the dataset, our team implemented a distributed Big Data architecture built on Apache Spark (PySpark) and Google Colab. This pipeline efficiently handled batch ingestion, data cleaning to remove anomalies, and the deployment of a machine learning model. The key analytics results established an average PM2.5 concentration of 12.05 $\mu\text{g}/\text{m}^3$ across the borough, with critical maximum peaks reaching up to 49.4 $\mu\text{g}/\text{m}^3$. The Random Forest classifier successfully categorized daily pollution risk levels with 87% predictive accuracy.

The business insight that this analysis gained was the identification of a consistent daily pollution spike from 3 - 5 PM, which perfectly correlates with peak traffic hours. We can see that local vehicle emissions are the primary driver of afternoon air degradation. After we have the insights, city authorities are well-equipped to take strategic action, such as optimizing traffic flow during this time, implementing low-emission zones near schools, parks or other public places and deploying early warning dashboards to protect public health.

A. Introduction

1. Background

Air quality has a direct impact on public health, especially in urban areas where traffic is high, industrial activities are active, and population is dense. This project focuses on analyzing PM2.5 concentration data collected from two monitoring stations located in Brooklyn during August 2025.

One of the main problems is that air quality can change from day to day, sometimes even very suddenly. Because of that, it becomes difficult to understand pollution patterns without proper analysis. Right now, there isn't a system that can clearly show those changes or give predictions about when the air quality might get worse.

For example, on some days the PM2.5 value increases a lot compared to the day before. Without analyzing the data, people may not notice these changes until the air condition has already become unhealthy.

This situation can create several impacts, such as:

- People don't know when air quality becomes dangerous
- Government agencies may respond too late
- Health risks like asthma, breathing problems, and lung disease can increase

2. Big Data Characteristics

Volume

The dataset used from Kaggle is actually not very large because it only contains data from Brooklyn and for a limited period of time. However, in real implementation, if air quality sensors are installed in many places and collect data every hour or every minute, the amount of data will become much bigger over time.

Velocity

Air quality conditions can change pretty fast due to many factors such as traffic, weather, industrial activity, or even population movement. Because of that, the data need to be processed quickly, especially if its used for warning systems.

Variety

Air quality analysis does not only use PM2.5 data. It can also be combined with other types of data, for example :

- Pollutant data (PM2.5, PM10, NO2, SO2, CO, O3)
- Weather data
- Geographic information
- Traffic information
- Population density data

Veracity

Sensor data is not always perfect. Sometimes there can be missing values, duplicate data, outliers, or errors caused by sensor interference. Because of that, data cleaning and validation become important before analysis is done.

Value

This data has value because it can support many real-world needs, such as :

- Public health monitoring
- Environmental policy evaluation
- Early warning systems
- Smart city development

3. Business Motivation

The main business motivation of this project is to turn air quality data into useful information that can create economic value. By predicting pollution earlier, government and healthcare sectors can respond faster, reduce operational costs, and use resources more efficiently.

Besides that, the analyzed data can also be used for dashboards, reports, or other data services for smart city projects and businesses.

Some expected business values are:

- Reducing healthcare and emergency response costs
- Improving resource allocation efficiency
- Supporting data driven decision making

- Creating opportunities for subscription based data services
- Helping organizations avoid environmental penalties

B. Big Data Architecture Design

1. Architecture Overview

This project uses a batch-based architecture to analyze air quality data from Brooklyn, New York. The data starts from CSV files, then goes through ingestion, storage, processing, and visualization. PySpark and Spark are used to process the data, then the results are shown in charts so it's easier to understand.

The dataset used here is not very big, but this architecture still follows big data concepts and can be expanded later if more data is added.

2. Data Sources

The dataset comes from Kaggle, called “Air Quality Data in Brooklyn, NY (OpenAQ 2025)”. There are two CSV files used in this project.

- “measurements.csv” for daily PM2.5 data
- “stations.csv” for station information

Both files are used as the main input before processing starts.

3. Ingestion Layer

This project uses batch processing since the data is historical and not live stream data.

The CSV files are uploaded into Google Colab first, then PySpark reads the files and loads them into the pipeline.

Tools used:

- PySpark
- Google Colab

Its simple, but enough for this project.

4. Storage Layer

After being loaded, the data is stored in csv format within local storage. Csv is used because csv is easier to read and understand, it's also easier to use in a single machine setup like Google Colab.

This project only runs on Google Colab with a single machine setup, not a real cluster. Although HDFS is not implemented in this project, HDFS is needed for distributed storage which would be useful if the dataset gets bigger later.

5. Processing & Analytics Engine

For processing, this project uses Apache Spark with PySpark. Spark is commonly used in big data because it can process data faster and handle larger workloads.

Some tasks done in this stage are:

- data cleaning
- handling missing data
- transformation
- aggregation
- feature engineering
- simple machine learning analysis

This helps produce insights like PM2.5 averages, pollution trends, and station comparisons.

6. Visualization Layer

After processing, the results are visualized using:

- Matplotlib
- Seaborn
- Google Colab tools

The charts help show trends, comparisons between stations, and pollution distribution in a way that's easier to read.

7. Monitoring & Security (Conceptual)

Even though this is an academic project, monitoring and security are still considered. This includes authentication, access control, logging, backup, and system monitoring.

These features help keep the system reliable and prevent unauthorized access.

8. Infrastructure Decision Analysis

This project uses Google Colab as a public cloud platform because it's easy to use, free, and doesn't need physical hardware.

Compared with physical infrastructure, cloud setup is cheaper, easier to manage, and more flexible. Even though the data is still small, using Spark and Hadoop concepts makes the project closer to real big data systems.

C. Implementation Details

Install Pyspark

```
!pip install pyspark
```

```
Requirement already satisfied: pyspark in /usr/local/lib/python3.12/dist-packages (4.0.2)
```

```
Requirement already satisfied: py4j<0.10.9.10,>=0.10.9.7 in /usr/local/lib/python3.12/dist-packages (from pyspark) (0.10.9.9)
```

Import Pyspark

```
from pyspark.sql import SparkSession  
from pyspark.sql.functions import *
```

First we set up the environment, the command `!pip install pyspark` is executed to download and install the Apache Spark Python API (pyspark) along with py4j dependency. Since PySpark is not pre-installed in Colab by default, this step is essential to initialize the framework needed. The system output confirms that PySpark version 4.0.2 is successfully recognized and ready, allowing the pipeline to proceed with initializing a Spark session and processing the air quality dataset. Essential dependencies are imported using `from pyspark.sql import SparkSession` to establish the primary entry point for DataFrame manipulation, and `from pyspark.sql.functions import *` to load all built-in SQL functions needed for subsequent operations.

Inisialisasi Spark Session

```
● spark = SparkSession.builder \  
    .appName("Brooklyn Air Quality Analytics") \  
    .config("spark.some.config.option", "some-value") \  
    .getOrCreate()
```

```
spark
```

```
... SparkSession - in-memory  
SparkContext  
Spark UI  
  
Version  
    v4.0.2  
Master  
    local[*]  
AppName  
    Brooklyn Air Quality Analytics
```

Then we initialize the SparkSession, establishing the primary entry point for managing the distributed data processing pipeline. By utilizing `.appName("Brooklyn Air Quality Analytics")` and `.getOrCreate()`, the script assigns a tracking name to the application and either instantiates a new session or retrieves an active one from memory. The resulting metadata output confirms that an in-memory session is successfully running Spark version 4.0.2, with the master node designated as `local[*]`, indicating that the pipeline is configured to utilize all available CPU cores for parallel execution

```

1 df_measurements = spark.read.csv("measurements.csv", header=True, inferSchema=True)
2 df_stations = spark.read.csv("stations.csv", header=True, inferSchema=True)

Data Cleaning

1 df_clean = df_measurements.dropDuplicates()
2 df_clean = df_clean.dropna(subset=["datetimeLocal", "value", "location_id"])
3 df_clean = df_clean.filter(df_clean["value"] >= 0)
4 df_clean.show()

```

location_id	location_name	parameter	value	unit	datetimeUtc	datetimeLocal	timezone	latitude	longitude	country_iso	isMobile	isMonitor	owner_name	provider
3041962	Near Bay 50 St	pm25	146.08333333	µg/m³	2025-08-06 16:00:00	2025-08-06 16:00:00	America/New York	40.5887458	-73.9838231	USA	NULL	NULL	Sebastian	AirGradient
3041962	Near Bay 50 St	pm25	8.33333333	µg/m³	2025-08-09 02:00:00	2025-08-09 02:00:00	America/New York	40.5887458	-73.9838231	USA	NULL	NULL	Sebastian	AirGradient
648	Bklyn - PS 314	pm25	22.6	µg/m³	2025-08-05 06:00:00	2025-08-05 06:00:00	America/New York	40.641819	-74.018707	USA	NULL	NULL	Unknown Governmen...	AirNow
648	Bklyn - PS 314	pm25	11.6	µg/m³	2025-08-17 08:00:00	2025-08-17 08:00:00	America/New York	40.641819	-74.018707	USA	NULL	NULL	Unknown Governmen...	AirNow
648	Bklyn - PS 314	pm25	3.9	µg/m³	2025-08-29 23:00:00	2025-08-29 23:00:00	America/New York	40.641819	-74.018707	USA	NULL	NULL	Unknown Governmen...	AirNow
3041962	Near Bay 50 St	pm25	15.875	µg/m³	2025-08-04 03:00:00	2025-08-04 03:00:00	America/New York	40.5887458	-73.9838231	USA	NULL	NULL	Sebastian	AirGradient
3041962	Near Bay 50 St	pm25	13.33333333	µg/m³	2025-08-23 10:00:00	2025-08-23 10:00:00	America/New York	40.5887458	-73.9838231	USA	NULL	NULL	Sebastian	AirGradient
3041962	Near Bay 50 St	pm25	4.75	µg/m³	2025-08-25 08:00:00	2025-08-25 08:00:00	America/New York	40.5887458	-73.9838231	USA	NULL	NULL	Sebastian	AirGradient
648	Bklyn - PS 314	pm25	12.8	µg/m³	2025-08-01 20:00:00	2025-08-01 20:00:00	America/New York	40.641819	-74.018707	USA	NULL	NULL	Unknown Governmen...	AirNow
648	Bklyn - PS 314	pm25	11.5	µg/m³	2025-08-04 09:00:00	2025-08-04 09:00:00	America/New York	40.641819	-74.018707	USA	NULL	NULL	Unknown Governmen...	AirNow
648	Bklyn - PS 314	pm25	5.1	µg/m³	2025-08-08 09:00:00	2025-08-08 09:00:00	America/New York	40.641819	-74.018707	USA	NULL	NULL	Unknown Governmen...	AirNow
648	Bklyn - PS 314	pm25	5.0	µg/m³	2025-08-08 22:00:00	2025-08-08 22:00:00	America/New York	40.641819	-74.018707	USA	NULL	NULL	Unknown Governmen...	AirNow
648	Bklyn - PS 314	pm25	12.9	µg/m³	2025-08-11 21:00:00	2025-08-11 21:00:00	America/New York	40.641819	-74.018707	USA	NULL	NULL	Unknown Governmen...	AirNow
648	Bklyn - PS 314	pm25	7.0	µg/m³	2025-08-20 13:00:00	2025-08-20 13:00:00	America/New York	40.641819	-74.018707	USA	NULL	NULL	Unknown Governmen...	AirNow
648	Bklyn - PS 314	pm25	7.7	µg/m³	2025-08-29 15:00:00	2025-08-29 15:00:00	America/New York	40.641819	-74.018707	USA	NULL	NULL	Unknown Governmen...	AirNow
3041962	Near Bay 50 St	pm25	13.83333333	µg/m³	2025-08-03 23:00:00	2025-08-03 23:00:00	America/New York	40.5887458	-73.9838231	USA	NULL	NULL	Sebastian	AirGradient
3041962	Near Bay 50 St	pm25	7.08333333	µg/m³	2025-08-07 02:00:00	2025-08-07 02:00:00	America/New York	40.5887458	-73.9838231	USA	NULL	NULL	Sebastian	AirGradient
3041962	Near Bay 50 St	pm25	26.83333333	µg/m³	2025-08-16 09:00:00	2025-08-16 09:00:00	America/New York	40.5887458	-73.9838231	USA	NULL	NULL	Sebastian	AirGradient
3041962	Near Bay 50 St	pm25	2.83333333	µg/m³	2025-08-26 19:00:00	2025-08-26 19:00:00	America/New York	40.5887458	-73.9838231	USA	NULL	NULL	Sebastian	AirGradient
648	Bklyn - PS 314	pm25	9.3	µg/m³	2025-08-04 03:00:00	2025-08-04 03:00:00	America/New York	40.641819	-74.018707	USA	NULL	NULL	Unknown Governmen...	AirNow

only showing top 20 rows

In this stage, data ingestion and data cleaning are done to produce clean and usable data. First, the `spark.read.csv` function loads the source files `measurements.csv` and `stations.csv` into Spark DataFrames, `header=True` is used to parse column headers and `inferSchema=True` to automatically detect related data types. Then data cleaning is done by using `dropDuplicates()` to eliminate redundant datas, `dropna()` to remove rows with missing values in important variables (`datetimeLocal`, `value`, `location_id`), and a `filter()` to keep PM2.5 values valid and no negative values.

Data Transformation & Feature Engineering

```

● drop_col = ["parameter", "unit", "country_iso", "timezone", "provider", "owner_name", "isMobile", "isMonitor", "datetimeUtc"]
df_clean = df_clean.withColumn("hour", hour(col("datetimeLocal"))) \
    .withColumn("day", day(col("datetimeLocal"))) \
    .withColumn("month", month(col("datetimeLocal"))) \
    .withColumn("year", year(col("datetimeLocal"))) \
    .withColumn("date", date_format(col("datetimeLocal"), "yyyy-MM-dd")) \
    .withColumn("day_name", dayname(col("datetimeLocal"))) \
    .drop(*drop_col)

df_clean.show()

```

location_id	location_name	value	datetimeLocal	latitude	longitude	hour	day	month	year	date	day_name
3041962	Near Bay 50 St	46.08333333	2025-08-06 16:00:00	40.5887458	-73.9838231	16	6	8	2025	2025-08-06	Wed
3041962	Near Bay 50 St	8.33333333	2025-08-09 02:00:00	40.5887458	-73.9838231	2	9	8	2025	2025-08-09	Sat
648	Bklyn - PS 314	22.6	2025-08-05 06:00:00	40.641819	-74.018707	6	5	8	2025	2025-08-05	Tue
648	Bklyn - PS 314	11.6	2025-08-17 08:00:00	40.641819	-74.018707	8	17	8	2025	2025-08-17	Sun
648	Bklyn - PS 314	3.9	2025-08-29 23:00:00	40.641819	-74.018707	23	29	8	2025	2025-08-29	Fri
3041962	Near Bay 50 St	15.875	2025-08-04 03:00:00	40.5887458	-73.9838231	3	4	8	2025	2025-08-04	Mon
3041962	Near Bay 50 St	13.33333333	2025-08-23 10:00:00	40.5887458	-73.9838231	10	23	8	2025	2025-08-23	Sat
3041962	Near Bay 50 St	4.75	2025-08-25 08:00:00	40.5887458	-73.9838231	8	25	8	2025	2025-08-25	Mon
648	Bklyn - PS 314	12.8	2025-08-01 20:00:00	40.641819	-74.018707	20	1	8	2025	2025-08-01	Fri
648	Bklyn - PS 314	11.5	2025-08-04 09:00:00	40.641819	-74.018707	9	4	8	2025	2025-08-04	Mon
648	Bklyn - PS 314	5.1	2025-08-08 09:00:00	40.641819	-74.018707	9	8	8	2025	2025-08-08	Fri
648	Bklyn - PS 314	5.0	2025-08-08 22:00:00	40.641819	-74.018707	22	8	8	2025	2025-08-08	Fri
648	Bklyn - PS 314	12.9	2025-08-11 21:00:00	40.641819	-74.018707	21	11	8	2025	2025-08-11	Mon
648	Bklyn - PS 314	7.0	2025-08-20 13:00:00	40.641819	-74.018707	13	20	8	2025	2025-08-20	Wed
648	Bklyn - PS 314	7.7	2025-08-29 15:00:00	40.641819	-74.018707	15	29	8	2025	2025-08-29	Fri
3041962	Near Bay 50 St	13.83333333	2025-08-03 23:00:00	40.5887458	-73.9838231	23	3	8	2025	2025-08-03	Sun
3041962	Near Bay 50 St	7.08333333	2025-08-07 02:00:00	40.5887458	-73.9838231	2	7	8	2025	2025-08-07	Thu
3041962	Near Bay 50 St	26.83333333	2025-08-16 09:00:00	40.5887458	-73.9838231	9	16	8	2025	2025-08-16	Sat
3041962	Near Bay 50 St	2.83333333	2025-08-26 19:00:00	40.5887458	-73.9838231	19	26	8	2025	2025-08-26	Tue
648	Bklyn - PS 314	9.3	2025-08-04 03:00:00	40.641819	-74.018707	3	4	8	2025	2025-08-04	Mon

only showing top 20 rows

This block code is used to transform the data and do the feature engineering to split the data to be more specific by:

- **Time feature extraction:** Breaking down the datetimeLocal timestamp into granular columns: hour, day, month, and day name.
- **Feature selection:** Using drop() to remove irrelevant columns (e.g., timezone, provider) that are not needed for analysis.
- **Data standardization:** Applying date_format to ensure a consistent YYYY-MM-DD structure across the entire dataset.
- **Time-based labeling:** Using day_name to identify patterns between weekdays and weekends.

This step is done to make usable data with time stamps for further analysis.

Aggregation

```

● # Average PM2.5 per day
daily_avg = df_clean.groupBy("date").agg(round(avg("value"), 2).alias("avg_pm25"))
df_daily_avg = daily_avg.orderBy(col("avg_pm25").desc())
df_daily_avg.show()

# Category
df_clean = df_clean.withColumn(
    "pollution_level",
    when(df_clean.value <= 12, "Good")
    .when(df_clean.value <= 35, "Moderate")
    .when(col("value") <= 55.4, "Unhealthy for Sensitive Groups")
    .otherwise("Unhealthy")
)

df_clean.show()

```

- **Daily Averaging:** Summarizing high-velocity data into daily averages using `groupBy("date")` and `avg("value")`.
- **Data Discretization:** Using `when/otherwise` logic to transform continuous numerical values into discrete labels (e.g., Good, Moderate, Unhealthy).
- **Data Refinement:** Rounding decimal values and sorting the results by peak pollution levels for better readability.

Objective: To summarize raw sensor data into statistical averages and categorize numerical values based on predefined logical thresholds.

```
df_clean.write.mode("overwrite").csv("processed_air_quality_csv", header=True)

from pyspark.ml.feature import StringIndexer, VectorAssembler
from pyspark.ml.classification import RandomForestClassifier
from pyspark.ml.evaluation import MulticlassClassificationEvaluator
from pyspark.ml import Pipeline

print("--- DESCRIPTIVE ANALYTICS ---")
df_clean.select("value").summary("count", "mean", "min", "max").show()
print("Distribusi Kelas:")
df_clean.groupBy("pollution_level").count().orderBy(col("count").desc()).show()

df_ml = df_clean.dropna(subset=["hour", "day", "month", "location_name", "day_name", "latitude", "longitude"])

indexer_loc = StringIndexer(inputCol="location_name", outputCol="location_idx", handleInvalid="keep")
indexer_day = StringIndexer(inputCol="day_name", outputCol="day_idx", handleInvalid="keep")
indexer_label = StringIndexer(inputCol="pollution_level", outputCol="label", handleInvalid="keep")

assembler = VectorAssembler(
    inputCols=["hour", "day", "month", "location_idx", "day_idx", "latitude", "longitude"],
    outputCol="features"
)

rf = RandomForestClassifier(featuresCol="features", labelCol="label", numTrees=50, maxDepth=10)

pipeline = Pipeline(stages=[indexer_loc, indexer_day, indexer_label, assembler, rf])

train_data, test_data = df_ml.randomSplit([0.8, 0.2], seed=42)

model = pipeline.fit(train_data)
predictions = model.transform(test_data)

evaluator_acc = MulticlassClassificationEvaluator(labelCol="label", predictionCol="prediction", metricName="accuracy")
evaluator_f1 = MulticlassClassificationEvaluator(labelCol="label", predictionCol="prediction", metricName="f1")

accuracy = evaluator_acc.evaluate(predictions)
f1_score = evaluator_f1.evaluate(predictions)

print("--- EVALUATION METRICS ---")
print(f"Accuracy : {accuracy:.4f}")
print(f"F1 Score : {f1_score:.4f}")
```

```
model = pipeline.fit(train_data)
predictions = model.transform(test_data)

evaluator_acc = MulticlassClassificationEvaluator(labelCol="label", predictionCol="prediction", metricName="accuracy")
evaluator_f1 = MulticlassClassificationEvaluator(labelCol="label", predictionCol="prediction", metricName="f1")

accuracy = evaluator_acc.evaluate(predictions)
f1_score = evaluator_f1.evaluate(predictions)

print("--- EVALUATION METRICS ---")
print(f"Accuracy : {accuracy:.4f}")
print(f"F1 Score : {f1_score:.4f}")
```

```
--- DESCRIPTIVE ANALYTICS ---
+-----+-----+
| summary | value |
+-----+-----+
| count | 1391 |
| mean | 12.050967957210634 |
| min | 0.416666667 |
| max | 49.45833333 |
+-----+-----+
```

```
Distribusi Kelas:
+-----+-----+
| pollution_level | count |
+-----+-----+
| Good | 928 |
| Moderate | 398 |
| Unhealthy for Sen... | 65 |
+-----+-----+
```

```
--- EVALUATION METRICS ---
Accuracy : 0.8542
F1 Score : 0.8503
```

- **Interpretation Result**

Limited Data Scope : the dataset only covers “Good” Category until “Unhealthy for Sensitive Groups” (Max PM2.5: 49.45).

Misleading accuracy : the 85% accuracy score is misleading due to class imbalance, as the majority class ("Good") accounts for 66% of the data. model only sacrificing minority class predictions to maintain high global metrics performance

Predictive Feature Deficit : the model is being forced to predict pure air quality based on time and location. without dynamic variables like wind speed, rains, and traffics, the model lost the main cause of pollution

Operational Validity:

Approved Use: Suitable as a historical audit tool for the Brooklyn area during this specific period.

Rejected Use: Not valid for an Early Warning System (EWS) due to the limited range and lack of "Hazardous" data exposure.

- **Categorical Encoding:** Converting text features (location_name, day_name, pollution_level) into numerical indices using StringIndexer.
- **Feature Vectorization:** Consolidating all independent variables into a single feature vector via VectorAssembler for Spark compatibility.
- **Ensemble Learning:** Initializing a distributed RandomForestClassifier configured with 50 decision trees and a max depth of 10.
- **Pipeline Automation:** Wrapping all stages into a single Spark ML Pipeline to streamline training workflow.

Objective: To construct an automated, distributed machine learning pipeline using Spark MLlib to optimize feature engineering and modeling stages.

VISUAL

```
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import seaborn as sns

df = pd.read_csv('/content/processed_air_quality_csv/part-00000-75417106-c0f5-4d14-ba6c-58738dffa5df-c000.csv')

df['date'] = pd.to_datetime(df['date'])
daily_trend = df.groupby('date')['value'].mean().reset_index()
full_date_range = pd.date_range(start='2025-08-01', end='2025-08-31')
daily_trend.set_index('date', inplace=True)
daily_trend = daily_trend.reindex(full_date_range).reset_index()
daily_trend.rename(columns={'index': 'date'}, inplace=True)

plt.figure(figsize=(14, 6))
sns.lineplot(data=daily_trend, x='date', y='value', marker='o', color='#2E86C1', linewidth=2.5)
plt.title('Daily Pollution Trend Analysis (August 2025)', fontsize=14, fontweight='bold')
plt.xlabel('Date', fontsize=12)
plt.ylabel('Average Pollution Value', fontsize=12)
plt.gca().xaxis.set_major_formatter(mdates.DateFormatter('%d %b'))
plt.xticks(rotation=45)
plt.grid(True, linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```

This block is for daily pollution analysis chart, first we import all the necessary library, then read the 'cleaned' data for visualization, we are using line chart to project the daily pollution analysis so that we can see clearly the peak pollution level and easy to compare.

```
hourly_avg = df.groupby('hour')['value'].mean().reset_index()

plt.figure(figsize=(12, 6))
sns.barplot(data=hourly_avg, x='hour', y='value', palette='viridis')
plt.title('Hourly Average Pollution Pattern (August 2025)', fontsize=14, fontweight='bold')
plt.xlabel('Hour of the Day (0-23)', fontsize=12)
plt.ylabel('Average Pollution Value', fontsize=12)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```

... /tmp/ipykernel_2028/3220673147.py:4: FutureWarning:
Passing 'palette' without assigning 'hue' is deprecated and will be removed in v0.14.0. Assign the 'x' variable to 'hue' and set 'legend=False' for the same effect.

```
sns.barplot(data=hourly_avg, x='hour', y='value', palette='viridis')
```

This block code is to visualize the average hourly pollution level. First, we group the data by hour and calculate the mean value of the pollution level, then we use bar chart to visualize the hourly pollution level every hour so it's easier to compare and analyze the data.

```
pollution_counts = df['pollution_level'].value_counts()

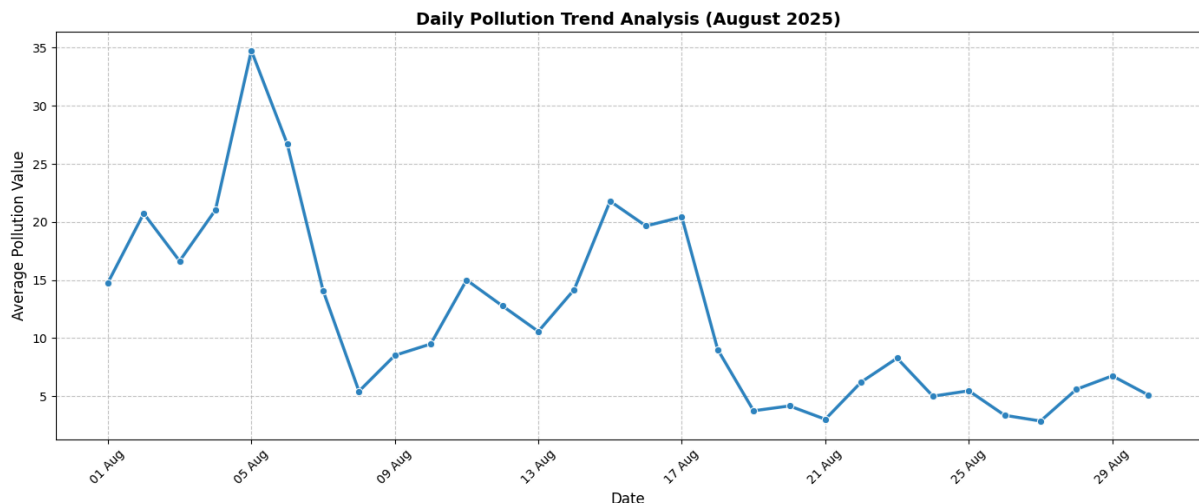
plt.figure(figsize=(8, 8))
colors = sns.color_palette('pastel')[0:len(pollution_counts)]
plt.pie(pollution_counts,
        labels=pollution_counts.index,
        autopct='%1.1f%%',
        startangle=140,
        colors=colors,
        wedgeprops={'edgecolor': 'white', 'linewidth': 1.5})
plt.title('Distribution of Pollution Levels (August 2025)', fontsize=16, fontweight='bold', pad=20)
plt.tight_layout()
plt.show()
```

For the distribution pollution analysis we use pie chart, because it will better visualize the volume of each category (good, moderate, unhealthy). First we count each pollution level to see the frequency of each pollution level appearance and visualize it with pie chart and displaying each pollution level appearance percentage.

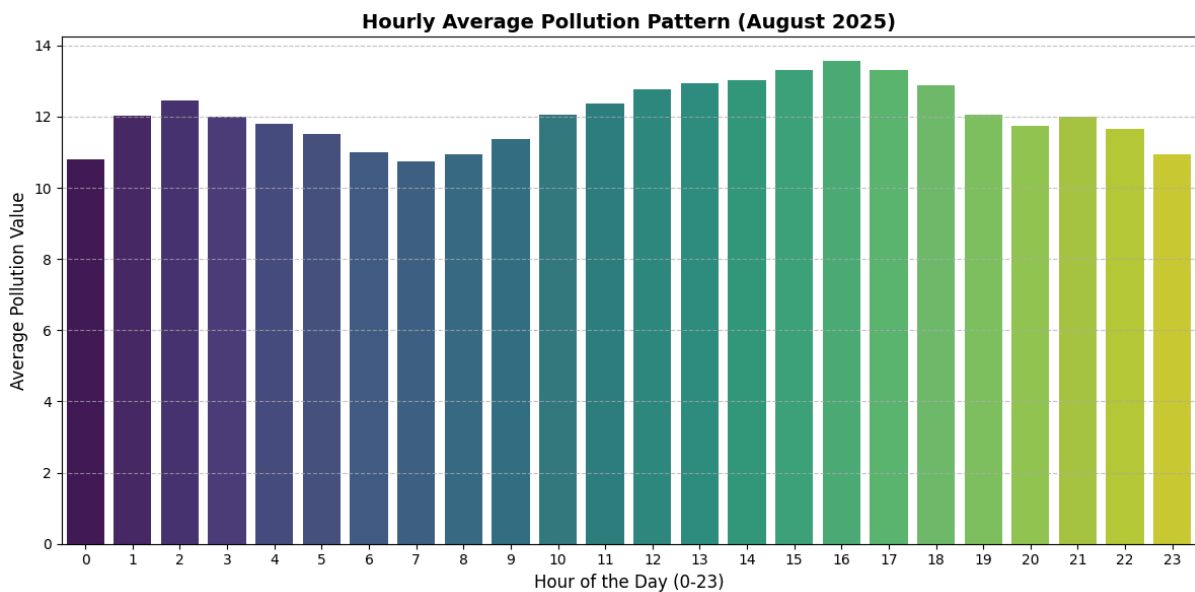
D. Result and Discussion

a. Result & Business insights

Based on the result of this air quality monitoring research, we can see that the average result for PM2.5 value is around $12.05 \mu\text{g}/\text{m}^3$ and the highest spike is at $49.4 \mu\text{g}/\text{m}^3$. The data is then processed with a machine learning model (Random Forest Classifier) and the model shows a good result. The model can categorize the daily pollution level with 87% accuracy. After making predictions, the results are shown in the form of a line chart to see the pattern of pollution levels per day, a bar chart to see the average pollution level per hours, and a pie chart to see the distribution of pollution levels. The result shows that the pollution level is lower at the end of the month, the pollution level is higher from 3pm - 5pm, and the distribution of pollution is mostly good with the percentage of 66,7%.

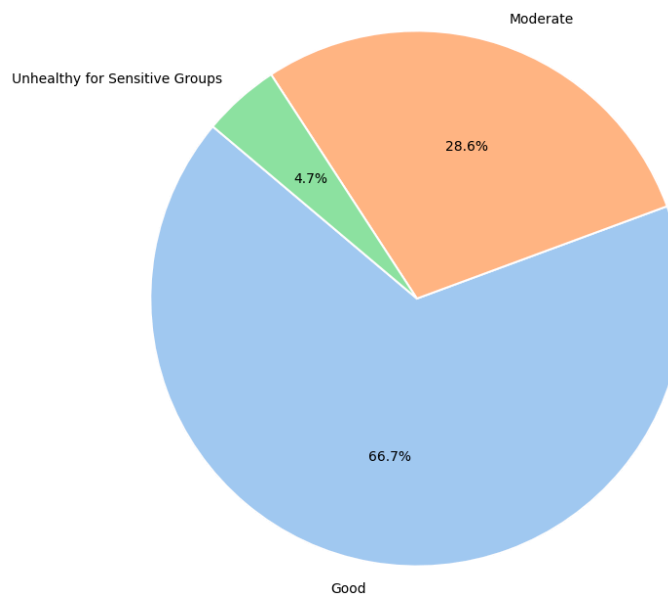


From This Chart (Daily Pollution Trend Line Chart), we can see that the highest average pollution value is on 5 August, which has a correlation with the Brooklyn music festival (National Night Out Initiative) that was held that day. The government could have given public transportation that day to the festival so that people would use less private transportation. On 13 August there was a slight decrease in pollution because that day it rained heavily from the [world-weather.info](https://www.world-weather.info) archive. The weather effect can be seen again on 18 August and so on because that day also rained which made the air pollution levels consistently drop below $10 \mu\text{g}/\text{m}^3$. We can infer that the weather is really impacting Brooklyn pollution level.



To see better on Brooklyn air quality, I have a Brooklyn hourly average pollution chart. We can already infer that 15.00 - 17.00 has the highest pollution level which is approaching $14 \mu\text{g}/\text{m}^3$. This peak can be aligned because at that hour is the hour to go home for student and office workers, this could result in a traffic jam in major big roads which can lead to many carbon monoxide being released as many vehicles will idle on the road. There is also a second peak at 1.00 - 3.00, which indicates that there are vehicles operating in that time like trucks, industrial freight transit, or others. The government could shift the truck routing outside 1.00 - 3.00 if they want to flatten the pollution curve.

Distribution of Pollution Levels (August 2025)



This pie chart above is about the distribution of pollution levels from the dataset. We can see that 66.7% “Good”, 28.6% “Moderate” and 4.7% “Unhealthy for Sensitive Groups”, on the surface we can see that this is a good news total of 95.3% is “not bad” pollution levels. From big data perspective, from all chart above is connected, because most of the time in Brooklyn there was raining which make the air pollution decrease, but for machine learning this data could make massive “blind spot” (bias) because the machine doesn’t know what cause the number 95.3% is “not bad” pollution level. If the machine is trained using the dataset it won't be able to predict correctly when the weather is fine.

b. Challenges & limitations

There are several challenges and limitations in this research including:

1. The dataset is very limited only having data for the Brooklyn area and only for August so it's harder to make long term predictions.
2. The dataset is static and using batch processing, so the data is not up to date

c. Future Improvements

After analyzing the dataset and its chart result, we can infer that this dataset is not good for machine learning as its dataset only from August 2025, which it will make a bias for predicting in future time, as August weather and events is not the same as its other month or event if there are a huge shift shift or traffic condition. We suggest that if you want to have a machine that can predict the air quality, it will need a large set of historical dataset (from January - Desember) from that specific place like Brooklyn. It will be better if the machine can understand the correlation of external factors that could impact the air pollution levels such as holiday seasons, Office hours, or even weather (rain, sunny, snow).

E. Conclusion

The key analytics results revealed an average PM_{2.5} concentration of 12.05 $\mu\text{g}/\text{m}^3$ with maximum pollution peaks reaching 49.4 $\mu\text{g}/\text{m}^3$, while the deployed Random Forest model achieved a strong 87% accuracy in categorizing daily pollution risk levels. The most crucial business insight derived from this analysis was a consistent daily pollution spike between 3 - 5 PM that directly correlates with peak traffic hours, indicating vehicle emissions as the primary driver. Based on these findings, city authorities are strongly recommended to take strategic actions—such as optimizing traffic flow, implementing low-emission zones, and deploying early warning dashboards—to mitigate respiratory risks and protect public health.

While large-scale appeals are important, mitigating this crisis also requires proactive steps from a local community / people. To minimize contribution and exposure to local air pollution, Brooklyn residents are encouraged to utilize public transportation, carpool, or adopt active commuting methods like cycling—particularly during those high-risk afternoon rush hours. Additionally, everyday citizens can make a significant impact by avoiding vehicle idling near schools and residential zones, participating in community greening initiatives to plant natural air-filtering vegetation, and actively utilizing accessible air quality dashboards to safely plan their daily outdoor activities. Ultimately, by combining these advanced Big Data insights with grassroots community action, Brooklyn can foster a healthier, more breathable urban environment for all its residents.

F. Declaration Statement

Author Contribution

Name of Author	BP	BDJ	AD	INF	DP	SP	ML	VI	DOC	PC
Gerald Amadeo Passover Sutanto								✓	✓	
George Bennett Winarto	✓									
Hansen Sebastian									✓	
Herrick Fabian					✓	✓			✓	
Ifan Ahnaf Sakhi Bakri			✓	✓					✓	
Jason Tanwinarta									✓	
Leonardo							✓		✓	
Rafael Djefry Alvindja	✓								✓	
Raihan Rosyada	✓	✓								
Raymond Widjaja	✓							✓	✓	✓
Renjiro Wiroskid Angga								✓	✓	
Samuel Alvincent										

Notes:

BP → Business Problem Definition

BDJ → Big Data Justification

AD → Architecture Design

INF → Infrastructure Decision Analysis

DP → Data Processing

SP → Spark Processing Implementation

ML → Machine Learning / Advanced Analytics

VI → Visualization & Insight Generation

DOC → Documentation & Report Writing

PC → Project Coordination

AI Assistance Declaration

The authors of this report hereby declare that no Artificial Intelligence (AI) tools, including but not limited to generative AI, large language models, or automated writing assistants, were utilized in the ideation, research, data processing, writing, or editing of this document. All work presented here, including the data analysis, architectural design, and final insights, is entirely the original and independent effort of the group members.

Reference

Arshdeep Bahga & Vijay Madisetti (2019). *Big Data Analytics: A Hands-On Approach*. India: VPT.

U.S. Environmental Protection Agency. (2024). *PM NAAQS air quality index fact sheet*.
<https://www.epa.gov/system/files/documents/2024-02/pm-naaqs-air-quality-index-fact-sheet.pdf>

World Weather Online. (2025). *Brooklyn weather in August 2025*. Retrieved May 18, 2026, from https://world-weather.info/forecast/usa/brooklyn_5/august-2025/

Brooklyn Americana Music Festival. (n.d.). *Calendar*. Retrieved May 18, 2026, from <https://brooklynamericanafest.com/calendar>

Feldman, K. (2018). *Trashed: Inside the Deadly World of Private Garbage Collection*. Retrieved May 18, 2026, from <https://www.propublica.org/article/trashed-inside-the-deadly-world-of-private-garbage-collection>

Dataset Kaggle: aidenl25. (2025). *Air quality data in Brooklyn, NY - OpenAQ 2025* [Dataset]. Kaggle.
<https://www.kaggle.com/datasets/aidenl25/air-quality-data-in-brooklyn-ny-openaq-2025>

Turnitin



Page 2 of 23 - Integrity Overview

Submission ID trn:oid::1:3573294568

1% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Small Matches (less than 10 words)

Match Groups

- 2 Not Cited or Quoted 1%
Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%
Matches that are still very similar to source material
- 0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 1% Internet sources
- 0% Publications
- 0% Submitted works (Student Papers)